

Universitatea POLITEHNICA din București
Facultatea de Electronică, Telecomunicații și Tehnologia Informației

**Analiza imaginilor cu fund de ochi pentru ajutor în
diagnosticul retinopatiei diabetice**

Lucrare de dizertație

Prezentată ca cerință parțială pentru obținerea
titlului de *Master*
în domeniul *Calculatoare și Tehnologia Informației*
programul de studii *Ingineria Informației*

Conducător științific
Conf. dr. ing. Laura Maria FLOREA

Absolvent
Stancovici Marian

Anul 2026

Declarație de onestitate academică

Prin prezenta declar că lucrarea cu titlul *Analiza imaginilor cu fund de ochi pentru ajutor în diagnosticul retinopatiei diabetice*, prezentată în cadrul Facultății de Electronică, Telecomunicații și Tehnologia Informației a Universității “Politehnica” din București ca cerință parțială pentru obținerea titlului de *Master* în domeniul Inginerie Electronică și Telecomunicații/ Calculatoare și Tehnologia Informației, programul de studii *Ingineria Informației* este scrisă de mine și nu a mai fost prezentată niciodată la o facultate sau instituție de învățământ superior din țară sau străinătate. Declar că toate sursele utilizate, inclusiv cele de pe Internet, sunt indicate în lucrare, ca referințe bibliografice. Fragmentele de text din alte surse, reproduse exact, chiar și în traducere proprie din altă limbă, sunt scrise între ghilimele și fac referință la sursă. Reformularea în cuvinte proprii a textelor scrise de către alți autori face referință la sursă. Înțeleg că plagiatul constituie infracțiune și se sancționează conform legilor în vigoare. Declar că toate rezultatele simulărilor, experimentelor și măsurărilor pe care le prezint ca fiind făcute de mine, precum și metodele prin care au fost obținute, sunt reale și provin din respectivele simulări, experimente și măsurători. Înțeleg că falsificarea datelor și rezultatelor constituie fraudă și se sancționează conform regulamentelor în vigoare.

București, Iunie 2026.

Absolvent: Stancovici Marian

.....

Cuprins

Lista figurilor	iv
Lista tabelelor	v
Lista acronimelor	vii
1. Fundamente teoretice - context medical și tehnic	5
1.1. Diabetul zaharat și complicațiile sale oculare	5
1.2. Anatomia ochiului și retina	5
1.3. Retinopatia diabetică — definiție, stadii de evoluție și simptome	5
1.4. Imagistica de fund de ochi ("fundus photography")	5
1.5. Diagnosticul clinic — procesul actual și limitările sale	5
2. Metode existente pentru detecția automată a retinopatiei diabetice	6
2.1. Analiza seturilor de date disponibile	6
2.2. Module dedicate	7
2.2.1. Timer	7
2.2.2. UART	7
2.2.3. SPI	7
2.2.4. GPIO	7
2.2.5. AXI RAM	7
2.3. Modulul de top și instanțierea SoC-ului	7
2.4. Logica de test și integrarea EDT	7
3. Metodologie și implementare	8
3.1. Sinteza circuitului	8
3.1.1. introducerea logicii DFT (lanuturi de SCAN)	8
3.1.2. Importarea librariilor si optimizare	8
3.2. Generarea vectorilor de test pentru defecte STUCK-AT și TRANSITION	8
3.2.1. Cazul fără EDT (fastscan)	8
3.2.2. Cazul cu EDT (variante cu 30, 40, 50, 60 de lanturi de SCAN)	8
3.3. Regenerarea EDT-ului și crearea netlist-ului pentru modulul de top	8
3.4. Simularea vectorilor de test și generarea testbench-ului	8
3.5. Etapa de diagnoză	8
4. Rezultate și discuții (ATPG)	9
4.1. Criterii de performanță ale sistemului pentru diferite variante de test (non-EDT vs EDT-30/40/50/60 chains)	9
4.1.1. Patternuri de test	9
4.1.2. Ciclii de shiftare	10
4.1.3. EDT Offset	10
4.1.4. Ciclii totali de shiftare (per pattern)	11

4.1.5.	Ciclii totali de shiftare	11
4.1.6.	Timp de testare	11
4.1.7.	Test Coverage	12
4.1.8.	Arie	12
5.	Concluzii	13
5.1.	Directii viitoare de cercetare si dezvoltare	13
5.2.	Contributii principale si concluzii	14
	Bibliografie	17
	Anexa A. Cod	18

Lista figurilor

4.1. Tabelul cu metrice de evaluare pentru generarea patternurilor de test	9
--	---

Lista tabelelor

Lista acronimelor

SOTA = State of the Art

Deep DRiD = Deep Diabetic Retinopathy Image Dataset

UoA-DR = University of Auckland Diabetic Retinopathy

Introducere

Contextul și motivația lucrării

Retinopatia diabetică reprezintă o provocare majoră de sănătate publică, fiind principala cauză a orbirii prevenibile în rândul populației active. Creșterea continuă a numărului de pacienți cu diabet generează o presiune imensă asupra sistemelor medicale, depășind adesea capacitatea de screening a specialiștilor. Deoarece afecțiunea poate evolua asimptomatic în fazele inițiale, identificarea pacienților la risc este critică înainte ca leziunile retiniene să devină ireversibile, subliniind necesitatea unor soluții de monitorizare scalabile și eficiente.

Complexitatea diagnosticării este dată de necesitatea unei clasificări precise, conform standardelor internaționale care împart boala în cinci clase de severitate [1], [2]. Procesul impune medicului să distingă între stadiile incipiente, marcate doar de microanevrisme discrete [3], și formele avansate, caracterizate de hemoragii și neovascularizație. Această granularitate a diagnosticului este dificilă de menținut constant în programele de screening în masă, unde oboseala și subiectivitatea factorului uman pot duce la erori de clasificare cu impact direct asupra deciziilor terapeutice.

Pentru a depăși aceste limitări, domeniul imagisticii medicale a adoptat rapid soluțiile bazate pe inteligență artificială. Trecerea de la metodele clasice de procesare a imaginii [4] la algoritmi de învățare profundă (Deep Learning) a marcat un punct de inflexiune, validat clinic prin studii extensive [5]. Rețelele neurale convoluționale au eliminat necesitatea definirii manuale a regulilor vizuale [6], având capacitatea de a învăța autonom trăsături complexe direct din datele brute [7] și de a identifica corelații subtile între texturi și patologii.

Evoluția recentă a acestor tehnologii a permis tranziția de la modele experimentale simple la arhitecturi robuste [8], capabile să opereze în condiții clinice reale. Cercetarea actuală s-a distanțat de clasificarea binară elementară, concentrându-se pe sisteme care pot gestiona variabilitatea imaginilor provenite din diverse surse prin tehnici de adaptare la domeniu [9]. Algoritmi moderni demonstrează o reziliență crescută la artefacte și zgomot vizual, oferind o consistență a diagnosticului care rivalizează cu expertiza umană [7], [10].

Obiectivele lucrării

Scopul lucrării de față este de a folosi algoritmi de vedere computerizată bazați pe rețele convoluționale adânci pentru a crea o metodă de detectare și clasificare a retinopatiei diabetice. Metoda se va testa pe seturi de date publice. Implementarea se va face în Python folosind biblioteci specifice.

Aceasta urmărește atingerea următoarelor obiective:

- Analizarea metodelor deja propuse în literatura de specialitate pentru detecția și clasificarea automată a retinopatiei diabetice folosind imagini cu fund de ochi.

- Căutarea seturilor de date publice ce conțin imagini cu fund de ochi și adnotări pentru clasificarea retinopatiei diabetice.
- Analizarea seturilor de date găsite.
- Propunerea unei metode bazate pe învățare profundă pentru detecția și clasificarea retinopatiei diabetice.
- Analizarea performanței pentru metoda propusă în diverse cazuri de antrenare/testare

Structura lucrării

Lucrarea de față este structurată în mai multe capitole, fiecare din ele abordând un subiect esențial din proiect:

- Capitolul 1: Introducere

În acest capitol se prezintă contextul și motivația lucrării, cât și cum se propune să se redacteze și să se structureze, menționând aplicabilitatea acestuia în lumea reală.

- Capitolul 2: Fundamente teoretice - context medical și tehnic

Acest capitol va oferi o privire de ansamblu asupra retinopatiei diabetice, explicând cauzele, simptomele și stadiile acestei afecțiuni, precum și importanța detecției și clasificării automate a acestuia. De asemenea, se vor introduce conceptele de bază ale învățării profunde și ale rețelelor neuronale convoluționale, care vor fi utilizate în dezvoltarea metodei propuse.

- Capitolul 3: Metode existente pentru detecția automată a retinopatiei diabetice

Acest capitol va oferi o analiză detaliată a metodelor existente pentru detecția și clasificarea retinopatiei diabetice, evidențiind avantajele și limitările acestora, precum și modul în care acestea au evoluat în timp.

- Capitolul 4: Metodologie și implementare

Acest capitol va detalia metodologia și implementarea propusă pentru detecția și clasificarea retinopatiei diabetice, explicând arhitectura rețelei neuronale utilizate, procesul de antrenare și optimizare, precum și modul în care aceasta se diferențiază de metodele existente.

- Capitolul 5: Metoda propusă

Acest capitol va detalia metoda propusă pentru detecția și clasificarea retinopatiei diabetice, explicând arhitectura rețelei neuronale utilizate, procesul de antrenare și optimizare, precum și modul în care aceasta se diferențiază de metodele existente.

- Capitolul 6: Rezultate experimentale și discuții

Acest capitol va prezenta rezultatele obținute în urma testării metodei propuse pe seturile de date alese, comparând performanța acestuia cu alte metode existente și discutând avantajele și limitările sale.

- Capitolul 7: Concluzii

Acest capitol va sintetiza concluziile generale ale lucrării, evidențiind contribuțiile aduse și impactul potențial al metodei propuse în domeniul detecției automate a retinopatiei diabetice.

Capitolul 1

Fundamente teoretice - context medical și tehnic

- 1.1 Diabetul zaharat și complicațiile sale oculare
- 1.2 Anatomia ochiului și retina
- 1.3 Retinopatia diabetică — definiție, stadii de evoluție și simptome
- 1.4 Imagistica de fund de ochi ("fundus photography")
- 1.5 Diagnosticul clinic — procesul actual și limitările sale

Capitolul 2

Metode existente pentru detecția automată a retinopatiei diabetice

2.1 Analiza seturilor de date disponibile

Performanța și capacitatea de generalizare a modelelor de învățare profundă sunt intrinsec dependente de calitatea, volumul și diversitatea datelor utilizate în procesul de antrenare. În contextul oftalmologiei computaționale, disponibilitatea bazelor de date publice a permis tranziția de la metodele clasice de procesare a imaginii la arhitecturi neuronale complexe, oferind un teren comun pentru evaluarea obiectivă a algoritmilor.

- **Seturi de referință pentru clasificarea globală (“Grading”)**

- **EyePACS** (Kaggle Diabetic Retinopathy Detection) [5], [7] rămâne cea mai vastă resursă publică disponibilă, conținând aproximativ 88.702 de imagini. Importanța sa strategică constă în eterogenitatea extremă a imaginilor (variații de expunere, focalizare, artefacte), fiind critic pentru antrenarea unor rețele backbone robuste capabile să gestioneze fenomenul de “domain shift” [9] și să generalizeze pe imagini de calitate slabă
- **APTOS 2019** Blindness Detection [7] reprezintă standardul modern de calitate. Colectat de Aravind Eye Hospital, acest set de 3.662 de imagini este utilizat frecvent în literatura recentă pentru “fine-tuning”, oferind o vizibilitate superioară a detaliilor retiniene comparativ cu seturile mai vechi
- **MESSIDOR-2** [7] este considerat în prezent „Benchmark-ul de Aur” pentru testarea algoritmilor. Extinzând setul original Messidor la 1.748 de imagini, acesta se distinge prin consistența diagnostică și este utilizat ca domeniu țintă (Target Domain) în studiile de adaptare [9] și pentru validarea sistemelor explicabile (XAI) [7], oferind o metrică fidelă a performanței reale pe date curate.
- **DeepDRiD** [11] aduce o inovație necesară prin structurarea celor 2.000 de imagini în perechi binoculare și includerea notelor de calitate, simulând fluxul de triaj clinic.
- **UoA-DR** [7] este o resursă istorică (200 imagini), utilizată complementar în studiile comparative pentru a asigura diversitatea demografică a validării.

- **Seturi specializate pentru segmentarea leziunilor și analiza detaliată**

—

•

2.2 Module dedicate

2.2.1 Timer

2.2.2 UART

2.2.3 SPI

2.2.4 GPIO

2.2.5 AXI RAM

2.3 Modulul de top și instanțierea SoC-ului

2.4 Logica de test și integrarea EDT

Capitolul 3

Metodologie și implementare

3.1 Sinteza circuitului

3.1.1 introducerea logicii DFT (lanuturi de SCAN)

3.1.2 Importarea librariilor si optimizare

3.2 Generarea vectorilor de test pentru defecte STUCK-AT și TRANSITION

3.2.1 Cazul fără EDT (fastscan)

3.2.2 Cazul cu EDT (variante cu 30, 40, 50, 60 de lanturi de SCAN)

3.3 Regenerarea EDT-ului și crearea netlist-ului pentru modulul de top

3.4 Simularea vectorilor de test și generarea testbench-ului

3.5 Etapa de diagnoză

Capitolul 4

Rezultate și discuții (ATPG)

4.1 Criterii de performanță ale sistemului pentru diferite variante de test (non-EDT vs EDT-30/40/50/60 chains)

În acest capitol se vor sintetiza toate informațiile discutate la capitolele anterioare, atât despre sinteză cât și pe partea de generare de vectori de test pentru toate cazurile, și se vor analiza și discuta rezultatele obținute, făcându-se o comparație obiectivă. Următoarea figură prezintă toate rezultatele comprimate într-un tabel cu metrici de performanță.

ATPG										
Final	Stuckat					Transition				
	FastScan	EDT 30 chains	EDT 40 chains	EDT 50 chains	EDT 60 chains	FastScan	EDT 30 chains	EDT 40 chains	EDT 50 chains	EDT 60 chains
Patternuri totale	883	1122	1253	1398	1575	1609	2492	3158	3485	3805
Ciclii de shiftare	5548	185	139	112	93	5548	185	139	111	93
Offset de EDT	0	26	26	26	26	0	26	26	27	26
Ciclii totali de shiftare (per pattern)	5548	211	165	138	119	5548	211	165	138	119
Ciclii totali de shiftare	4898884	236742	206745	192924	187425	8926732	525812	521070	480930	452795
test time (@100MHz) [ms]	48.989	2.367	2.067	1.929	1.874	89.267	5.258	5.211	4.809	4.528
test coverage (%)	99.78	99.74	99.73	99.66	99.57	98.76	98.71	98.66	98.58	98.44
Arie (μm^2)	41526	42068	42117	42220	42295	41526	42068	42117	42220	42295

Figura 4.1: Tabelul cu metrici de evaluare pentru generarea patternurilor de test

Rezultatele s-au obținut pe urma simulărilor de generare de vectori de test pentru ATPG normal, respectiv pentru ATPG folosindu-se EDT cu 30,40,50 sau 60 de lanțuri de scan, aceste simulări realizându-se pentru a detecta defecte de tip stuck-at și transition.

În continuare se va lua pe rând fiecare metrică și se va explica utilitatea acesteia și rezultatele ei.

4.1.1 Patternuri de test

După cum se poate observa, în cazul ATPG-ului normal s-au generat 883 de vectori de test (acest număr nu se poate anticipa), în timp ce, integrând modulul de EDT, crește considerabil numărul de vectori de test generați. Acest lucru se întâmplă din următoarele cauze:

- EDT-ul, așa cum se specifică și în denumirea acestuia, se concentrează pe metode deterministice de generare de vectori de test care vizează defecte și modele de defecte specifice. Acestea sunt mai eficiente decât metodele aleatoare, dar tind să genereze un număr mai mare de vectori de test.
- EDT-ul este conceput să detecteze defecte greu de detectat cu metode de ATPG tradiționale, realizând acest lucru prin adăugarea de logică de test suplimentară în circuit pentru a crește observabilitatea și controlabilitatea nodurilor interne. Acest lucru necesită mai multe patternuri de test pentru un test-coverage cât mai mare.

- Diferența în numărul de lanțuri de scan.

În cazul fără EDT există un singur lanț de scan, în timp ce, adăugând modulul de EDT, se pot configura mai multe lanțuri de scan prin care datele se deplasează în paralel, ajungând la mai multe patternuri de test.

De asemenea, numărul de vectori de test se mărește și în cazul în care se crește numărul lanțurilor de scan pentru EDT. Cu cât numărul acestora este mai mare, cu atât scade gradual lungimea fiecărui lanț de scan și numărul de flip-flopi prin care circulă datele este distribuit mai multor lanțuri, ceea ce produce efectul descris.

Cât despre tipul de defect, există câteva motive principale de ce este nevoie de un număr mult mai mare de vectori de test pentru a detecta defecte de tip transition decât cele de tip stuck-at:

- **Propagarea defectului:** Defectele de tranziție au nevoie ca vectorul de test respectiv să poată propaga efectul până într-un punct observabil din circuit fără a se pierde între timp, în comparație cu defectele stuck-at care doar blochează intrarea sau ieșirea unei porți la o valoare fixă.
- **Considerente de sincronizare:** Defectele transition sunt mult mai sensibile la sincronizarea tranziției semnalelor de la o valoare în alta, ceea ce conduce la mai mulți vectori de test pentru a putea capta timpii necesari detectării defectului.

4.1.2 Ciclii de shiftare

Un ciclu de shiftare reprezintă o deplasare a datelor cu o poziție în lanțul de scan corespunzător. Cum în cazul ATPG normal este prezent un singur lanț de scan, numărul ciclilor este egal cu dimensiunea lanțului respectiv deoarece datele circulă prin toți flip-flopii de scan înainte să fie măsurați la ieșire.

În restul cazurilor (EDT), numărul total de celule de scan se împarte la numărul scan chain-urilor și se obține numărul de ciclii de ceas pentru un lanț de scan, datele circulând în paralel. Exemplu pentru EDT cu 30 de lanțuri de scan:

$$\text{ciclii shiftare} = \frac{\text{număr total de celule de scan}}{\text{număr lanțuri de scan}} = \frac{5548}{30} = 185$$

Tipul de defect nu afectează acest număr deoarece logica de testare din interiorul modulului principal nu se modifică.

4.1.3 EDT Offset

Din perspectiva ATE-ului, design-ul prezintă un singur lanț de scan care circulă prin intrarea `edt_channels_in` înainte să intre în decompresor.

Acest aspect se întâmplă doar în cazul EDT-ului, iar fiecare pattern necesită un număr de ciclii de shiftare adiționali care se adună la numărul de patternuri din fiecare lanț calculat la punctul anterior.

Acești ciclii adiționali provin din modul de funcționare și structura EDT-ului, mai exact din [?]:

- ciclii de initializare
- biții de mascare (dacă se folosește o tehnologie specializată numită Xpress)

- biți de low-power (când se folosește un decompresor de putere scăzută)
- biții de pipeline definiți de utilizator

Numărul de cicluri de inițializare se poate calcula cu ajutorul următoarei formule [?] :

$$\text{număr cicluri de inițializare} = \lceil \frac{\text{dimensiunea decompresorului}}{\text{numărul de canale}} \rceil$$

, unde fracția se rotunjește la valoarea sa superioară. De exemplu, dacă avem un design în care decompresorul are dimensiunea de 32 de biți și există 6 canale de scan, iar numărul de celule de scan din fiecare lanț este 870, atunci este nevoie de 6 cicluri de inițializare. Fiecare lanț având 870 de celule și fiecare vector comprimat având nevoie de 6 cicluri de inițializare, rezultă că este nevoie de 876 de deplasări per pattern (nefolosind ceilalți biți).

Acest număr se adună în continuare celorlalte elemente din lista de mai sus, rezultând astfel numărul total de cicluri de shiftare suplimentari:

cicluri suplimentari = cicluri de inițializare + biți de mascare + biți de low-power + biți de pipeline

În cazul nostru, din rapoartele de simulare reiese că există 26 de cicluri suplimentari, care se vor aduna la lungimea lanțului de scan, rezultând numărul total de cicluri de shiftare.

4.1.4 Cicli totali de shiftare (per pattern)

Așa cum s-a specificat anterior, numărul de cicluri totali de shiftare reprezintă suma dintre numărul de cicluri propriu-zis și offset-ul de EDT, în cazul ATPG-ului obișnuit acesta este 0.

4.1.5 Cicli totali de shiftare

Folosind numărul de cicluri de shift per pattern și numărul total de patternuri de test, putem calcula numărul total de cicluri de shift pentru întreg modulul cu ajutorul următoarei formule:

$$\text{cicluri totali de shift} = \text{cicluri de shift per pattern} \times \text{număr de patternuri}$$

Cu ajutorul acestei metrici se poate determina dimensiunea memoriei necesare pentru efectuarea simulărilor de generare de patternuri.

4.1.6 Timp de testare

Cum fiecare element din lanțurile de scan reprezintă flip-flipi de scan care sunt celule secvențiale de memorie, acestea funcționează în mod sincron cu ceasul sistemului. Prin urmare, folosindu-ne de frecvența sistemului putem calcula timpul necesar deplasării datelor prin lanțurile de scan folosind toate patternurile generate. Acesta se calculează cu formula:

$$\text{timp de testare} = \text{număr total de cicluri} \times \frac{1}{\text{frecvența ceasului de sistem}}$$

În cazul de față frecvența ceasului de sistem este 100MHz, ceea ce înseamnă că o perioadă de ceas are 0.01 μ s, adică 10 ns. De exemplu, pentru cazul cu 30 lanțuri de scan, avem 236.742 cicluri totali, ceea ce înseamnă că timpul total de testare este 2.367.420 ns, aproximativ 2,36 secunde.

De asemenea, având în vedere că datele circulă în paralel prin lanțuri, timpul de testare este mult mai mic în cazul utilizării EDT-ului în comparație cu ATPG normal (20 ori mai mic).

4.1.7 Test Coverage

În contextul design-for-testing (DFT), noțiunea de "test-coverage" se referă la măsura în care un set de teste aplicate unui sistem sau circuit electronic acoperă sau verifică toate părțile și funcționalitățile prevăzute. Acoperirea testelor este o metrică importantă a calității procesului de testare și al nivelului de încredere pe care ne putem baza ca să știm dacă produsul nu mai prezintă defecte după ce a fost testat.

După cum se poate observa din tabel, cu cât numărul de lanțuri de test este mai mare, cu atât test-coverage-ul este mai mic (putem spune că sunt mărimi invers proporționale), deoarece circuitul devine mai puțin controlabil din punct de vedere al testării. Cu alte cuvinte, un raport de compresie mai mare duce la mărirea compresiei pentru fiecare pattern de test, dar de asemenea crește și posibilitatea generării de patternuri care nu pot fi comprimate și care duc în final la un test-coverage mai mic.

4.1.8 Arie

Deși nu este o metrică atât de importantă pentru testare precum test-coverage-ul, am vrut să evidențiez 2 aspecte:

- Logica EDT adaugă o porțiune foarte mică de hardware suplimentar în circuit.
În acest caz având un modul cu doar 64000 de porți logice, logica suplimentară introdusă este de aproximativ 1%, însă pentru un chip în curs de testare care prezintă milioane de porți, logica introdusă pentru EDT ajunge undeva la 0,35% din arie, ceea ce este o adiție infimă în comparație cu timpul de test câștigat.
- Cu cât creștem numărul de lanțuri de scan, cu atât crește aria folosită, însă foarte puțin în comparație cu aria totală, coverage-ul scăzând și el deoarece circuitul este din ce în ce mai greu de testat.

Capitolul 5

Concluzii

5.1 Direcții viitoare de cercetare și dezvoltare

Cu toate ca această lucrare reprezintă un pas important în implementarea și testarea circuitelor digitale pentru detectarea defectelor de fabricație folosind vectori de test generați automat, există multiple direcții viitoare de cercetare și dezvoltare care pot îmbunătăți și extinde rezultatele obținute. Acestea includ următoarele, dar nu se limitează la acestea:

- **Optimizarea performanței sistemului de testare:** Un aspect esențial care poate fi explorat este optimizarea performanței sistemului de testare. În mod specific, se pot studia metode avansate de reducere a timpului de testare fără a compromite acuratețea și acoperirea testelor. Algoritmi noi și tehnici de optimizare a scan chain-urilor și a vectorilor de test pot contribui semnificativ la îmbunătățirea eficienței procesului de testare.
- **Integrarea noilor tehnologii de testare:** Pe măsură ce tehnologia avansează, noi metode și instrumente de testare devin disponibile. Integrarea acestor tehnologii, precum tehnici avansate de Built-In Self-Test (BIST) sau utilizarea inteligenței artificiale pentru generarea și analiza vectorilor de test, poate îmbunătăți semnificativ capacitatea de detectare a defectelor și eficiența procesului de testare.
- **Extinderea scalabilității:** Un alt domeniu de cercetare este extinderea scalabilității sistemului de testare pentru circuite mai complexe și mari. Explorarea modului în care tehnologiile actuale de SCAN și EDT pot fi adaptate și extinse pentru system-on-chips (SoC) de dimensiuni mai mari și cu complexitate sporită poate duce la îmbunătățiri importante în detectarea defectelor.
- **Dezvoltarea de metode de testare pentru noi tipuri de defecte:** În timp ce lucrarea de față se concentrează pe detectarea defectelor de tip STUCKAT și TRANSITION, există și alte tipuri de defecte care pot afecta performanța circuitelor digitale. Cercetarea și dezvoltarea unor metode de testare pentru detectarea defectelor de tip delay, crosstalk și defecte intermitente poate asigura o acoperire mai completă și o fiabilitate crescută a circuitelor testate.
- **Implementarea și validarea în cazuri practice:** O direcție importantă de dezvoltare este aplicarea și validarea metodologiilor propuse în contexte industriale reale. Colaborarea cu industria semiconductorilor pentru implementarea soluțiilor dezvoltate în cadrul unor proiecte pilot poate oferi feedback valoros și poate ajuta la rafinarea tehnicilor de testare.
- **Automatizarea procesului de diagnoză:** Deși lucrarea a abordat deja etapa de diagnoză, există loc pentru automatizarea completă a acestui proces. Dezvoltarea unor instrumente de diagnoză automată, care pot identifica rapid și precis locația defectelor în circuit, ar contribui semnificativ la reducerea timpului de depanare și la îmbunătățirea eficienței generale a procesului de testare.

- **Studii comparative asupra metodelor de testare și posibilitatea combinării acestora:** Realizarea unor studii comparative între diferitele tehnologii și metode de testare disponibile poate oferi o înțelegere mai profundă a avantajelor și dezavantajelor fiecărei abordări. Aceste studii pot ghida alegerea metodologiilor optime pentru diferite tipuri de circuite și aplicații.

De asemenea, se poate aborda în viitor o posibilă combinare a acestor metode pentru o automatizare și o eficientizare mai puternică, extrăgând punctele forte din fiecare metodă de testare.

Deci, deși s-au atins obiective importante în domeniul detectării defectelor de fabricație în circuitele digitale, există multiple direcții viitoare de cercetare și dezvoltare care pot contribui la avansarea cunoștințelor și tehnologiilor în acest domeniu. Explorarea acestor direcții poate duce la sisteme de testare mai eficiente, fiabile și scalabile, asigurând astfel calitatea și performanța superioară a circuitelor digitale.

5.2 Contribuții principale și concluzii

Lucrarea de față, intitulată "Detectia defectelor de fabricație într-un circuit digital cu ajutorul vectorilor de test generați automat", a avut ca scop implementarea unui set de teste pentru un circuit digital, pentru depistarea defectelor de fabricație (DFT). Circuitul digital, descris în limbaj Verilog, a constat dintr-un SoC bazat pe tehnologie RISC-V, cu module dedicate de Timer, UART, SPI și periferice GPIO. În acest capitol se vor sintetiza concluziile principale obținute în urma cercetării și implementării efectuate.

În cadrul acestei lucrări am utilizat tehnologii complexe de testare precum tehnologia SCAN (introducerea lanțurilor de scan în circuit + transformarea flip-flopilor în celule de scan), ATPG (generarea automata a vectorilor de test cu ajutorul constrângerilor), DFT (concept care reunește noțiunile de SCAN, ATPG, EDT) și EDT (compresia vectorilor de test pentru îmbunătățirea performanțelor). Fiecare din aceste tehnologii a prezentat un rol esențial în îmbunătățirea detectării defectelor și optimizarea procesului de testare.

Introducerea logicii de scan în circuit, generarea vectorilor (patternurilor) de test, compresia acestora, procesul de simulare și diagnoză, pentru defecte de tip STUCKAT și TRANSITION au reprezentat exemple și aspecte esențiale în asigurarea acurateții și eficienței testării.

Unul dintre principalele obiective majore ale lucrării a fost verificarea performanțelor sistemului, în special timpul de testare și test-coverage-ul utilizând diverse structuri de test folosind generarea automată de vectori de test. În acest context, s-au analizat performanțele testării modulului atât în cazul utilizării unui singur lanț de scan, cât și în cazul utilizării multiplelor lanțuri de scan cu ajutorul tehnologiei EDT.

Rezultatele au demonstrat că utilizarea EDT a redus semnificativ timpul de testare, păstrând în același timp test-coverage-ul aproape de valoarea rezultată testării cu un singur lanț de scan, iar testele realizate cu 30, 40, 50, respectiv 60 de lanțuri de scan au arătat că este nevoie de atenție atunci când se alege acest număr, păstrând raportul dintre timpul de testare și test coverage în balanță.

Procesul de sinteză a inclus nu doar interpretarea și importarea librăriilor pentru celulele de bază ale circuitului (porți logice, bistabile etc), cât și inserarea logicii de scan în circuit. Acesta s-a realizat prin scripturi bine definite și cu ajutorul makefile-urilor pentru a păstra ordinea și automatizarea în execuția scripturilor.

S-a efectuat analiza codului sursă HDL și al librăriilor (pentru verificarea erorilor de sintaxă și semantică din codul realizat), elaborarea design-ului (transformarea acestuia într-un format inteligibil pentru tool-ul Design Compiler), compilarea design-ului (transformarea design-ului într-o reprezentare structurală), inserția logicii de scan, cât și generarea netlist-ului sintetizat gata pentru etapa de ATPG, pe care am discutat-o anterior, în care am generat vectorii de test pentru structuri de test cu și fără EDT cu ajutorul tool-ului Tessent - Mentor Graphics.

Simularea patternurilor de test folosind SimVision a permis vizualizarea semnalelor de scan (scan_enable, scan_mode, cât și edt_channels_in, edt_channels_out, edt_reset și edt_update), și a flip-flopilor de scan interconectate în lanțuri de scan (propagarea datelor în mod secvențial prin acestea).

Acest pas a fost crucial pentru a asigura că toți vectorii de test generați detectează toate defectele vizate în mod corect și într-un timp cât mai scurt.

În etapa de diagnoză, introducerea unui defect în mod intenționat în circuit a permis testarea capacității sistemului de a identifica exact poarta logică unde se regăsește defectul. Aceasta s-a depistat cu succes cu o precizie și un indice de încredere de 100%.

Această etapă a demonstrat că metodologia de testare utilizată este robustă și capabilă să localizeze cu acuratețe defectele din circuit, confirmând astfel validitatea și eficiența abordării propuse.

Lucrarea a demonstrat că utilizarea vectorilor de test generați automat și a tehnologiilor avansate de testare, precum EDT, poate îmbunătăți semnificativ detectarea defectelor de fabricație în circuitele digitale. Implementarea și testarea efectuate au arătat că abordarea propusă nu doar că reduce timpul de testare, dar și poate păstra test-coverage-ul la o valoare acceptabilă în același timp. În concluzie, metodologia aplicată în această lucrare oferă o soluție eficientă și scalabilă pentru detectarea defectelor în circuitele digitale complexe, fiind relevantă și aplicabilă în industria semiconductoarelor.

Bibliografie

- [1] Xiaofei Wang, Mai Xu, Jicong Zhang, Lai Jiang, and Liu Li. Deep multi-task learning for diabetic retinopathy grading in fundus images. In *Proceedings of the AAAI conference on artificial intelligence*, volume 35, pages 2826–2834, 2021.
- [2] Hossein Shakibania, Sina Raoufi, Behnam Pourafkham, Hassan Khotanlou, and Muharram Mansoorizadeh. Dual branch deep learning network for detection and stage grading of diabetic retinopathy. *Biomedical Signal Processing and Control*, 93:106168, 2024.
- [3] Joes Staal, Michael D Abràmoff, Meindert Niemeijer, Max A Viergever, and Bram Van Ginneken. Ridge-based vessel segmentation in color images of the retina. *IEEE transactions on medical imaging*, 23(4):501–509, 2004.
- [4] Akara Sopharak, Bunyarit Uyyanonvara, and Sarah Barman. Automatic exudate detection from non-dilated diabetic retinopathy retinal images using fuzzy c-means clustering. *sensors*, 9(3):2148–2161, 2009.
- [5] Varun Gulshan, Lily Peng, Marc Coram, Martin C Stumpe, Derek Wu, Arunachalam Narayanaswamy, Subhashini Venugopalan, Kasumi Widner, Tom Madams, Jorge Cuadros, et al. Development and validation of a deep learning algorithm for detection of diabetic retinopathy in retinal fundus photographs. *jama*, 316(22):2402–2410, 2016.
- [6] Harry Pratt, Frans Coenen, Deborah M Broadbent, Simon P Harding, and Yalin Zheng. Convolutional neural networks for diabetic retinopathy. *Procedia computer science*, 90:200–205, 2016.
- [7] Mohamed Chetoui and Moulay A Akhloufi. Explainable end-to-end deep learning for diabetic retinopathy detection across multiple datasets. *Journal of Medical Imaging*, 7(4):044503–044503, 2020.
- [8] Shiqi Huang, Jianan Li, Yuze Xiao, Ning Shen, and Tingfa Xu. Rtnet: relation transformer network for diabetic retinopathy multi-lesion segmentation. *IEEE Transactions on Medical Imaging*, 41(6):1596–1607, 2022.
- [9] Guanghua Zhang, Bin Sun, Zhaoxia Zhang, Jing Pan, Weihua Yang, and Yunfang Liu. Multi-model domain adaptation for diabetic retinopathy classification. *Frontiers in Physiology*, 13:918929, 2022.
- [10] Muhammad Kashif Jabbar, Jianzhuo Yan, Hongxia Xu, Zaka Ur Rehman, and Ayesha Jabbar. Transfer learning-based model for diabetic retinopathy diagnosis using retinal images. *Brain Sciences*, 12(5):535, 2022.
- [11] Ruhan Liu, Xiangning Wang, Qiang Wu, Ling Dai, Xi Fang, Tao Yan, Jaemin Son, Shiqi Tang, Jiang Li, Zijian Gao, et al. Deepdrid: Diabetic retinopathy—grading and image quality estimation challenge. *Patterns*, 3(6), 2022.

Anexa A

Cod

```

1 add_scan_groups grp1 dummy_design.testproc
2 add_scan_chains chain1 grp1      edt_si[0]      edt_so[0]
3 add_scan_chains chain2 grp1      edt_si[1]      edt_so[1]
4 add_scan_chains chain3 grp1      edt_si[2]      edt_so[2]
5 add_scan_chains chain4 grp1      edt_si[3]      edt_so[3]
6 add_scan_chains chain5 grp1      edt_si[4]      edt_so[4]
7 add_scan_chains chain6 grp1      edt_si[5]      edt_so[5]
8 add_scan_chains chain7 grp1      edt_si[6]      edt_so[6]
9 add_scan_chains chain8 grp1      edt_si[7]      edt_so[7]
10 add_scan_chains chain9 grp1      edt_si[8]      edt_so[8]
11 add_scan_chains chain10 grp1     edt_si[9]      edt_so[9]
12 add_scan_chains chain11 grp1     edt_si[10]     edt_so[10]
13 add_scan_chains chain12 grp1     edt_si[11]     edt_so[11]
14 add_scan_chains chain13 grp1     edt_si[12]     edt_so[12]
15 add_scan_chains chain14 grp1     edt_si[13]     edt_so[13]
16 add_scan_chains chain15 grp1     edt_si[14]     edt_so[14]
17 add_scan_chains chain16 grp1     edt_si[15]     edt_so[15]
18 add_scan_chains chain17 grp1     edt_si[16]     edt_so[16]
19 add_scan_chains chain18 grp1     edt_si[17]     edt_so[17]
20 add_scan_chains chain19 grp1     edt_si[18]     edt_so[18]
21 add_scan_chains chain20 grp1     edt_si[19]     edt_so[19]
22 add_scan_chains chain21 grp1     edt_si[20]     edt_so[20]
23 add_scan_chains chain22 grp1     edt_si[21]     edt_so[21]
24 add_scan_chains chain23 grp1     edt_si[22]     edt_so[22]
25 add_scan_chains chain24 grp1     edt_si[23]     edt_so[23]
26 add_scan_chains chain25 grp1     edt_si[24]     edt_so[24]
27 add_scan_chains chain26 grp1     edt_si[25]     edt_so[25]
28 add_scan_chains chain27 grp1     edt_si[26]     edt_so[26]
29 add_scan_chains chain28 grp1     edt_si[27]     edt_so[27]
30 add_scan_chains chain29 grp1     edt_si[28]     edt_so[28]
31 add_scan_chains chain30 grp1     edt_si[29]     edt_so[29]
32 add_scan_chains chain31 grp1     edt_si[30]     edt_so[30]
33 add_scan_chains chain32 grp1     edt_si[31]     edt_so[31]
34 add_scan_chains chain33 grp1     edt_si[32]     edt_so[32]
35 add_scan_chains chain34 grp1     edt_si[33]     edt_so[33]
36 add_scan_chains chain35 grp1     edt_si[34]     edt_so[34]
37 add_scan_chains chain36 grp1     edt_si[35]     edt_so[35]
38 add_scan_chains chain37 grp1     edt_si[36]     edt_so[36]
39 add_scan_chains chain38 grp1     edt_si[37]     edt_so[37]
40 add_scan_chains chain39 grp1     edt_si[38]     edt_so[38]
41 add_scan_chains chain40 grp1     edt_si[39]     edt_so[39]
42 add_scan_chains chain41 grp1     edt_si[40]     edt_so[40]
43 add_scan_chains chain42 grp1     edt_si[41]     edt_so[41]
44 add_scan_chains chain43 grp1     edt_si[42]     edt_so[42]
45 add_scan_chains chain44 grp1     edt_si[43]     edt_so[43]
46 add_scan_chains chain45 grp1     edt_si[44]     edt_so[44]
47 add_scan_chains chain46 grp1     edt_si[45]     edt_so[45]
48 add_scan_chains chain47 grp1     edt_si[46]     edt_so[46]
49 add_scan_chains chain48 grp1     edt_si[47]     edt_so[47]
50 add_scan_chains chain49 grp1     edt_si[48]     edt_so[48]
51 add_scan_chains chain50 grp1     edt_si[49]     edt_so[49]
52 add_scan_chains chain51 grp1     edt_si[50]     edt_so[50]
53 add_scan_chains chain52 grp1     edt_si[51]     edt_so[51]
54 add_scan_chains chain53 grp1     edt_si[52]     edt_so[52]
55 add_scan_chains chain54 grp1     edt_si[53]     edt_so[53]
56 add_scan_chains chain55 grp1     edt_si[54]     edt_so[54]
57 add_scan_chains chain56 grp1     edt_si[55]     edt_so[55]
58 add_scan_chains chain57 grp1     edt_si[56]     edt_so[56]
59 add_scan_chains chain58 grp1     edt_si[57]     edt_so[57]
60 add_scan_chains chain59 grp1     edt_si[58]     edt_so[58]
61 add_scan_chains chain60 grp1     edt_si[59]     edt_so[59]
62 add_scan_chains chain61 grp1     edt_si[60]     edt_so[60]
63 add_scan_chains chain62 grp1     edt_si[61]     edt_so[61]

```

```

64 add_scan_chains chain63 grp1      edt_si[62]      edt_so[62]
65 add_scan_chains chain64 grp1      edt_si[63]      edt_so[63]
66 add_scan_chains chain65 grp1      edt_si[64]      edt_so[64]
67 add_scan_chains chain66 grp1      edt_si[65]      edt_so[65]
68 add_scan_chains chain67 grp1      edt_si[66]      edt_so[66]
69 add_scan_chains chain68 grp1      edt_si[67]      edt_so[67]
70 add_scan_chains chain69 grp1      edt_si[68]      edt_so[68]
71 add_scan_chains chain70 grp1      edt_si[69]      edt_so[69]
72 add_clocks 0 clk

```

Listing A.1: dummy_design.dofile

```

1  #!/bin/bash
2
3  # EDT regeneration:
4  # edit or create skeleton_input_file (chains working on falling and rising edge)
5
6  # !!!!!!!!!!!!! add module load dft-mentor/2022.3 if not loaded !!!!!!!!!!!!!!!!!!!!!
7  moduleunload dft-mentor
8  module load dft-mentor/2022.3
9
10 cleartool co -unres -nc skeleton_input_file
11 cleartool co -unres -nc dummy_design*
12 cleartool co -unres -nc edt_ip.dofile
13
14 echo -e "\nDon't forget to change number_of_chains variable here, in skeleton_input_file and
    in source/verilog/rtl/soc_student_edt.v !!!\n"
15
16 declare -i number_of_chains=70
17 echo -e "\nCreating skeleton_input_file for $number_of_chains chains...\n"
18 create_skeleton_design -i skeleton_input_file -o dummy_design
19
20
21 echo -e "\nExecuting edt_ip.dofile in 5 seconds...\n"
22 echo -e "5....."
23 sleep 1
24 echo -e "4...."
25 sleep 1
26 echo -e "3..."
27 sleep 1
28 echo -e "2.."
29 sleep 1
30 echo -e "1."
31 sleep 1
32 tesseract -shell -dofile edt_ip.dofile -replace -logfile edt_ip.log
33
34
35 # generated EDT RTL: gen/created_edt.v
36 # copy EDT RTL to source dir
37 echo -e "\nCheckout and copying generated RTL in my source folder for edt design later...\n"
38 cleartool co -unres -nc ../source/verilog/rtl/edt.v
39 echo -e "\nStill passed, continuing...\n"
40 cp gen/created_edt.v ../source/verilog/rtl/edt.v
41
42
43 # copy and process gen/created_edt.dofile
44 # !!!ATTENTION testpattern/edt/edt_atpg/edt_cfg.dofile will be overwritten
45 cleartool co -nc ../testpattern/edt/edt_"$number_of_chains"_chains/edt_atpg_"$number_of_chains"
    "/edt_cfg.dofile
46 cp -f gen/created_edt.dofile ../testpattern/edt/edt_"$number_of_chains"_chains/edt_atpg_"
    "$number_of_chains"/edt_cfg.dofile
47
48 echo -e "\nDeleting lines with add_clocks and the others...\n"
49
50 sed -i '/^add_scan_groups/d' ../testpattern/edt/edt_"$number_of_chains"_chains/edt_atpg_"
    "$number_of_chains"/edt_cfg.dofile
51 sed -i '/^add_scan_chains/d' ../testpattern/edt/edt_"$number_of_chains"_chains/edt_atpg_"
    "$number_of_chains"/edt_cfg.dofile
52 sed -i '/^add_clocks/d' ../testpattern/edt/edt_"$number_of_chains"_chains/edt_atpg_"
    "$number_of_chains"/edt_cfg.dofile
53 sed -i '/^add_pin_constraints/d' ../testpattern/edt/edt_"$number_of_chains"_chains/edt_atpg_"
    "$number_of_chains"/edt_cfg.dofile
54

```

```
55 echo -e "\nScript finished!\n"
56
57 echo -e "Now make synthesis on the new edt with $number_of_chains chains"
```

Listing A.2: Bash Script for generating EDT